

Hardware ERP

Project Overview

ER diagram · application flow · what every file is for

Generated 2026-09-22 from branch `feature/cr-086-reports` · schema at migration V56

73 PostgreSQL tables 45 tenant-scoped	56 Flyway migrations never edited, only added	762 Java files 30 packages · 70 controllers	352 TS/TSX files 25 modules · 70 pages
221 documented endpoints API_REGISTRY.md	48 permission codes 4 seeded roles	114 backend test classes 36 Testcontainers ITs	11 Playwright suites 297 assertions

Java 21 · Spring Boot 3.4.2 · PostgreSQL 16 · Flyway · React 18 · TypeScript · Vite 6 · Tailwind · shadcn/ui · Playwright

Contents

1. What this project is
2. Architecture at a glance
3. Application flow — login, JWT and tenant isolation (full-page diagram)
4. Application flow — core business documents (full-page diagram)
5. Entity-relationship diagram (full-page diagram) and table guide
6. Roles and permissions
7. Repository map — where everything lives
8. Backend: foundation files and the anatomy of one module
9. Frontend: foundation files and the anatomy of one module
10. Registries and documentation — the source of truth
11. Conventions that never bend
12. How to run and verify
13. Appendix A — tables not drawn in the ER diagram

1. What this project is

Hardware ERP is a business system for hardware shops in India. One shop is one *tenant*. Each tenant's staff sign in, keep their product catalogue and stock, buy from suppliers, sell to customers through quotations, sales orders, delivery challans and GST invoices, record payments and expenses, run customer projects with hired labour, and pull reports including GSTR-1.

It is **one Spring Boot application, one React application and one PostgreSQL database**. Many shops share that one database; every tenant-owned table carries a `tenant_id` column and every query filters by it, using the tenant taken from the signed-in user's JWT — never from anything the browser sends. This is the single most important design decision in the codebase (CR-016), and the request-flow diagram in section 3 shows exactly where it is enforced.

Not microservices. **Not** a database or schema per tenant. Module folders express boundaries, not deployment units. The project is in **maintenance and extension**, not greenfield: the original twelve-module build order (Auth → Supplier → Customer → Category → Brand → Product → Product Variant → Purchase → Inventory → Quotation → Invoice → Payment) was completed, and later change requests added sales orders, delivery challans, credit notes, projects, labour, expenses, coupons, notifications, reports and the platform console.

Who uses it

Shop owner	Creates staff accounts and roles, sets up branding, bank accounts and WhatsApp, watches the dashboard, runs reports.
Counter staff / salesperson	Quotations, invoices, customers, stock lookup — the fast path on a phone or a counter PC.
Purchase staff / inventory manager	Suppliers, purchases, bill import, stock adjustments, low-stock follow-up.
Accountant	Payments, purchase payments, expenses, receivables ageing, GST summary, GSTR-1.
Project manager	Customer jobs: materials issued from stock, labour attendance, expenses, profitability.
Platform operator	A separate cross-tenant console for incidents, feature flags, subscriptions and support tickets.

Two ways to ship the same code (CR-059)

	CLOUD	SELF_HOSTED
Profiles	<code>prod,cloud</code>	<code>prod,selfhosted</code>
Database	managed PostgreSQL (Supabase, as a plain endpoint — no vendor SDK)	PostgreSQL container on the shop's own machine
Entry point	Render (API) + Vercel (SPA), <code>render.yaml</code>	<code>docker compose -f docker-compose.selfhosted.yml up -d</code>
Subscription billing	live (Razorpay)	off, no tier caps
Refresh cookie	<code>Secure</code> mandatory	<code>Secure</code> off by default (LAN http)

DeploymentModeGuard refuses to start a self-hosted process that points at the managed database host, so one client's data can never land in the shared SaaS database.

2. Architecture at a glance

```
Browser / phone
  React 18 SPA (Vite build, served by Vercel or the self-hosted nginx)
  routes/ → layouts/ → modules/<name>/pages → services/xService.ts
                                     |
                                     | services/apiClient.ts (one Axios instance)
                                     | Authorization: Bearer <JWT in memory>
                                     ▼ HttpOnly refresh cookie

Spring Boot 3.4.2 monolith (/api/v1/...)
  RequestCorrelationFilter → RateLimitFilter → JwtAuthenticationFilter → @PreAuthorize(permission)
  controller → service (@Transactional, SecurityUtils.requireCurrentTenantId()) → repository (...
  ByTenantId)
  side effects inside the same transaction: document_sequence, stock_movement, activity_log
  after commit, @Async: notification (email / SMS / WhatsApp)
                                     ▼

PostgreSQL 16 – one schema, Flyway-managed, tenant_id on every tenant-owned table
External: SMTP or SendGrid · Twilio · WhatsApp Cloud API (per tenant) · Anthropic (optional) · Razorpay
(cloud only) · OpenStreetMap (address picker)
```

Package structure (locked)

```
com.hardware.erp
├─ common.{entity,dto,exception,web,activity,idempotency,sequence,security,util,validation,image}
├─ config
├─ security.{ratelimit,totp,captcha}
├─ <module>.{entity,repository,dto,mapper,service,service.impl,controller} ← 27 business packages

frontend/src
├─ main.tsx, App.tsx, index.css
├─ routes/ layouts/ services/ theme/ shared/{components,hooks,lib,types,constants,data}
├─ modules/<name>/{types,validation,services,constants,components,forms,pages} ← 25 modules
```

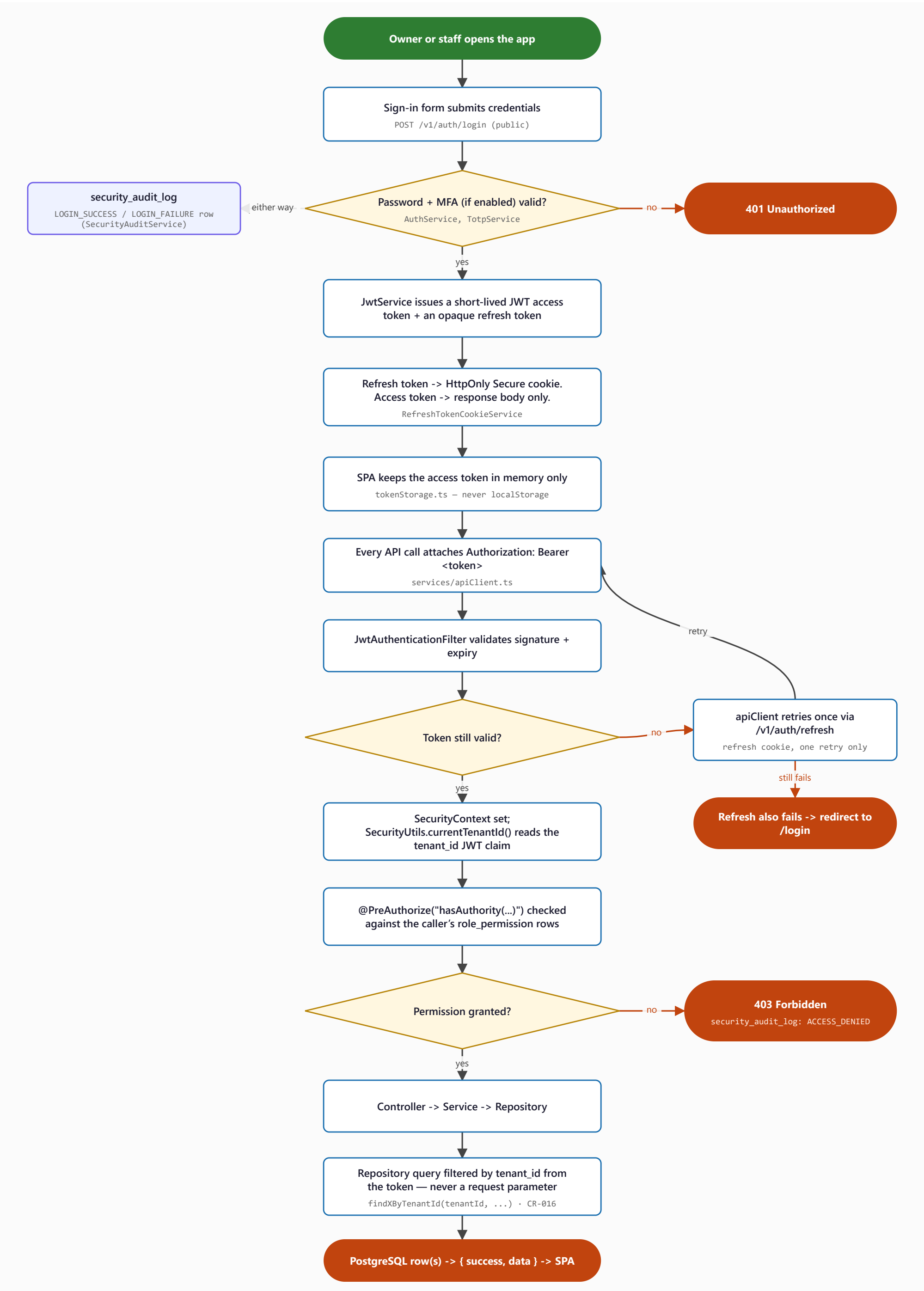
Module dependency graph (from the real imports)

invoice is the settlement point of the system and therefore a dependency magnet by design; it must never import upward. One known cycle exists and is recorded, not hidden: `product → invoice` for a product's recent sale prices (`ProductServiceImpl.priceHistory()`).

```
auth + tenant (every module below depends on both)
├─ supplier ── customer ── product ── coupon
├─ inventory ── product
├─ purchase ── supplier, product, inventory, invoice
├─ quotation ── customer, product, invoice
├─ invoice ── customer, product, inventory, coupon, notification
├─ salesorder ── customer, product, invoice, deliverychallan
├─ deliverychallan ── customer, product, inventory, invoice
├─ creditnote ── customer, product, inventory, invoice
├─ project ── customer, product, supplier, inventory, invoice, labour
├─ labour ── project, invoice
├─ expense, dashboard, analytics, report ── invoice
```

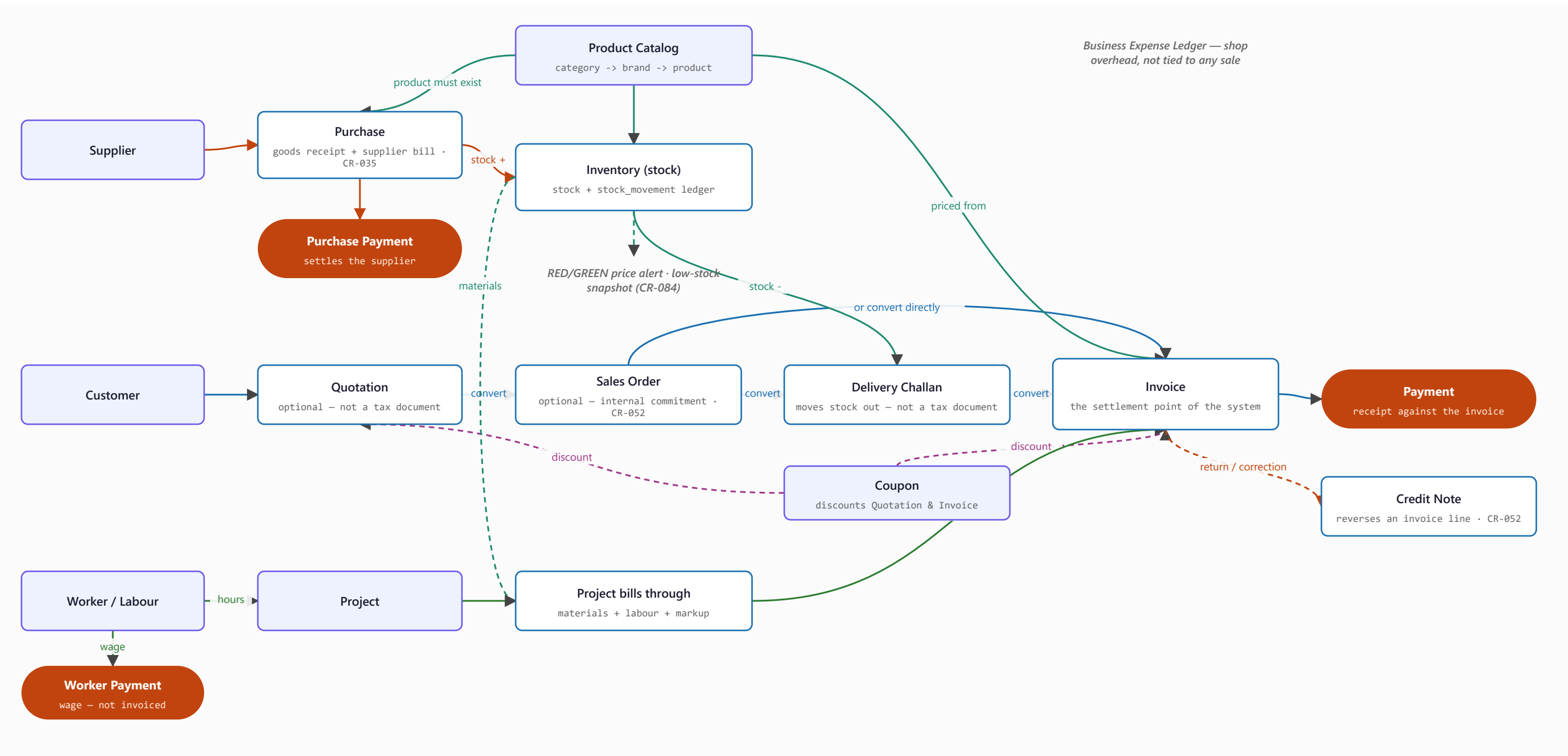
- └ ai (read-only tools), export (rendering only), notification, supportticket, legal, developer
- └ platformadmin — billing ← cross-tenant, deliberately OUTSIDE tenant isolation

3. Application flow — login, JWT and tenant isolation

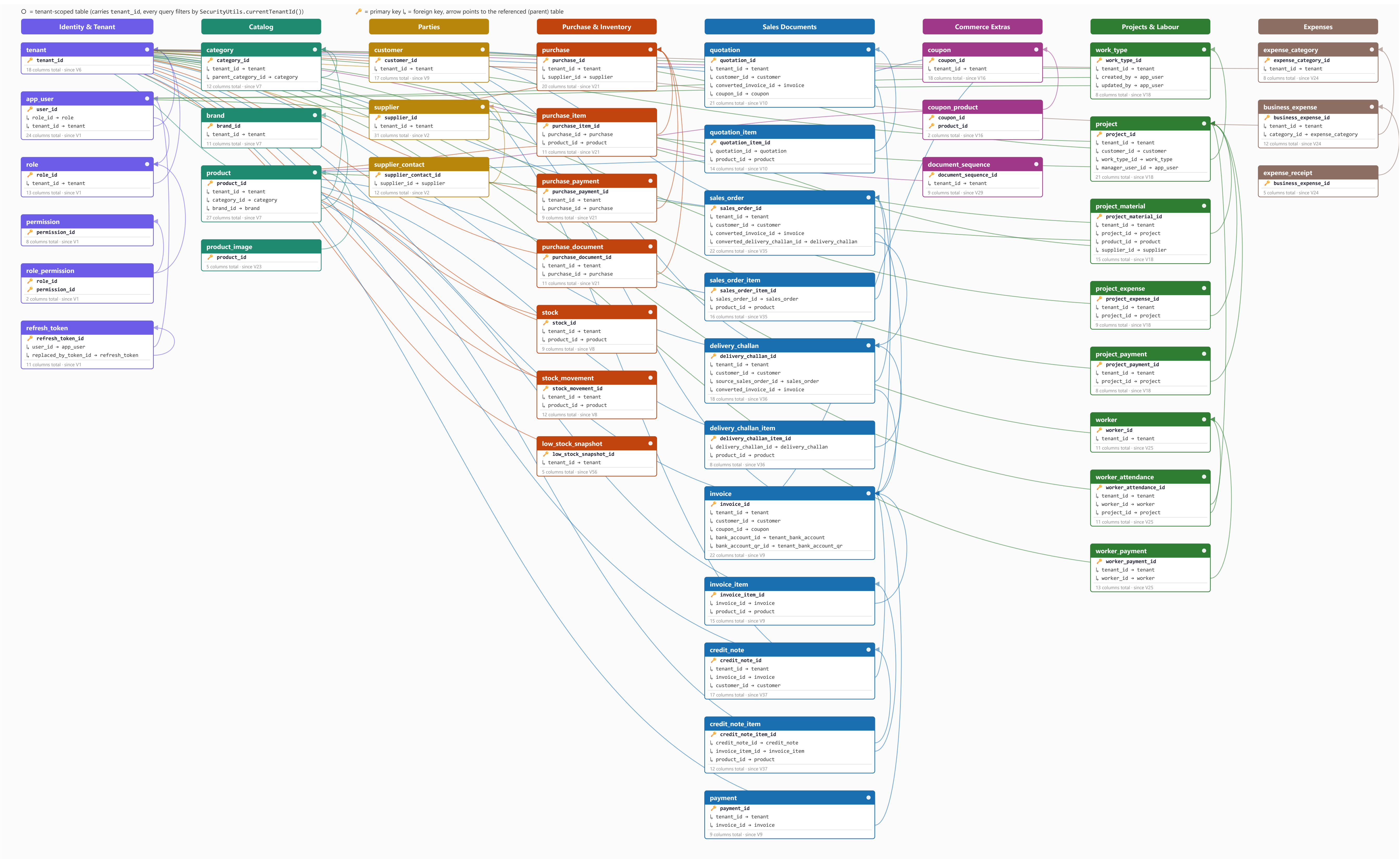


Read top to bottom. Red paths are refusals; the dashed line is the security audit trail. Files named in the boxes: `AuthController`, `AuthService`, `TotpService`, `JwtService`, `RefreshTokenCookieService`, `SecurityAuditService` (backend); `services/apiClient.ts`, `services/tokenStorage.ts` (frontend). The tenant is fixed at the JWT claim and used by every repository method; a request parameter can never change it (CR-016).

4. Application flow — core business documents



5. Entity-relationship diagram



45 of the 73 tables, grouped by business domain, with 90 foreign keys drawn. Each box lists its primary key and foreign keys only; the footer gives the full column count and the migration that created it. The remaining 28 tables (branding media, notifications, security audit, support desk, the platform console) are listed with their keys in Appendix A. Derived automatically from backend/src/main/resources/db/migration/V1..V56 — the diagram cannot drift from the schema. Line-item tables (*_item) carry no tenant_id of their own: they inherit scope through their parent document and are only ever loaded via it.

Foreign keys from drawn tables to appendix tables, not drawn to keep the picture readable: invoice.bank_account_id → tenant_bank_account, invoice.bank_account_qr_id → tenant_bank_account_qr.

5. Table guide — what each entity in the diagram holds

Same grouping and colours as the diagram. "Since" is the Flyway migration that created the table; later migrations may have added columns (the diagram footer counts them all).

■ Identity & Tenant

Table	Since	Cols	Tenant	What it holds
tenant	V6	18	yes	The shop. Name, GSTIN, address, subscription tier and trial expiry. Root of every tenant-scoped row.
app_user	V1	24	yes	A login account inside one tenant. Mobile/email are globally unique across tenants (CR-016 trade-off). Soft-deleted, never removed.
role	V1	13	yes	A named bundle of permissions per tenant. OWNER, MANAGER, ACCOUNTANT and STAFF are seeded as system roles; owners can add their own.
permission	V1	8	—	The 48 fine-grained permission codes (USER_MANAGE, INVOICE_CREATE, ...). The DB row is authoritative; PermissionCode.java mirrors it for the compiler.
role_permission	V1	2	—	Many-to-many join between role and permission.
refresh_token	V1	11	—	Opaque, rotating refresh tokens. replaced_by_token_id chains rotations so reuse of an old token is detected (BUG-AUTH-014).

■ Catalog

Table	Since	Cols	Tenant	What it holds
category	V7	12	yes	Product category tree (parent_category_id is self-referencing).
brand	V7	11	yes	Product brand.
product	V7	27	yes	One SKU: HSN code, GST rate, purchase and selling price in paise, unit, reorder level. category_id and brand_id are NOT NULL.
product_image	V23	5	—	Product photos stored as bytea (CR-036).

■ Parties

Table	Since	Cols	Tenant	What it holds
customer	V9	17	yes	Buyer. Credit limit in paise, GSTIN, addresses; the map picker (CR-076) fills the address.
supplier	V2	31	yes	Vendor. GSTIN, encrypted bank details (CR-018), soft-deleted with restore (CR-058).
supplier_contact	V2	12	—	Named contact people at a supplier.

■ Purchase & Inventory

Table	Since	Cols	Tenant	What it holds
purchase	V21	20	yes	A supplier bill / goods receipt. Created by hand or via the preview-then-confirm bill import (CR-035).
purchase_item	V21	11	—	One line of a purchase: product, quantity, unit rate DECIMAL(18,6).
purchase_payment	V21	9	yes	Money paid to the supplier against a purchase.
purchase_document	V21	11	yes	The scanned supplier bill attached to a purchase.
stock	V8	9	yes	Current on-hand quantity per product, per tenant.
stock_movement	V8	12	yes	Append-only ledger of every IN / OUT / ADJUST with its source document.
low_stock_snapshot	V56	5	yes	Daily count of products under reorder level, so the dashboard can show a real delta (CR-084).

■ Sales Documents

Table	Since	Cols	Tenant	What it holds
quotation	V10	21	yes	A priced offer to a customer. Not a tax document; converted_invoice_id links it to the invoice it became.
quotation_item	V10	14	—	One quotation line.
sales_order	V35	22	yes	Internal commitment after a quote is accepted. Converts to a delivery challan or straight to an invoice (CR-052).
sales_order_item	V35	16	—	One sales-order line, tracking delivered and invoiced quantities.
delivery_challan	V36	18	yes	Goods leave the shop. Moves stock, is not a tax document; converts to an invoice (CR-052).
delivery_challan_item	V36	8	—	One challan line.
invoice	V9	22	yes	The GST tax invoice — the settlement point of the whole system. Status UNPAID → PARTIALLY_PAID → PAID, or CANCELLED. Coupon, bank account and UPI QR are printed on the PDF.
invoice_item	V9	15	—	One invoice line with GST split.
credit_note	V37	17	yes	Return or correction against an invoice; puts stock back (CR-052).
credit_note_item	V37	12	—	One returned line, pointing at the original invoice_item.
payment	V9	9	yes	A receipt against an invoice: amount in paise and method (cash, UPI, bank...).

■ Commerce Extras

Table	Since	Cols	Tenant	What it holds
coupon	V16	18	yes	Discount code with validity window, usage limit and optional product restriction (CR-028).
coupon_product	V16	2	—	Restricts a coupon to particular products.

Table	Since	Cols	Tenant	What it holds
<code>document_sequence</code>	V29	9	yes	Per-tenant, per-document-type running numbers, allocated under SELECT ... FOR UPDATE so two invoices can never share a number (CR-041).

■ Projects & Labour

Table	Since	Cols	Tenant	What it holds
<code>work_type</code>	V18	8	yes	Kind of job a project is (e.g. rooftop installation).
<code>project</code>	V18	21	yes	A customer job with a manager, budget and computed profitability (CR-029).
<code>project_material</code>	V18	15	yes	Material issued to a project, from stock or bought from a supplier (CR-064 stock link).
<code>project_expense</code>	V18	9	yes	Cash spent on a project that is not material or labour.
<code>project_payment</code>	V18	8	yes	Money the customer paid against a project.
<code>worker</code>	V25	11	yes	A labourer on the shop's books (CR-036).
<code>worker_attendance</code>	V25	11	yes	One day of attendance, optionally charged to a project.
<code>worker_payment</code>	V25	13	yes	Wages and advances paid to a worker.

■ Expenses

Table	Since	Cols	Tenant	What it holds
<code>expense_category</code>	V24	8	yes	Owner-defined expense heads (rent, electricity...).
<code>business_expense</code>	V24	12	yes	A shop overhead entry — not tied to any sale or purchase.
<code>expense_receipt</code>	V24	5	—	Receipt image attached to an expense.

6. Roles and permissions

Authorisation is **permission-based, never role-based** (hard rule 6). A role is only a named set of permissions; business rules check a permission such as `INVOICE_CANCEL`, never "is the caller an owner". That is what lets an owner create a custom role — say "Senior counter" with `INVOICE_DISCOUNT_OVERRIDE` — without any code change.

Seeded role	Typical scope
<code>OWNER</code>	Everything in the shop, including users, roles, settings, subscription and data reset.
<code>MANAGER</code>	Day-to-day operations: products, stock, purchases, sales documents, projects, reports.
<code>ACCOUNTANT</code>	Invoices, payments, expenses, purchase payments, reports; no user or settings management.
<code>STAFF</code>	Counter work: quotations, invoices, customers, stock lookup.

The 48 permission codes live in the `permission` table (authoritative) and in `auth/entity/PermissionCode.java` (so `@PreAuthorize` strings are compiler-checked; `PermissionCodeConsistencyTest` asserts the two agree).

Families: USER, ROLE, AUDIT, CUSTOMER, SUPPLIER (incl. `SUPPLIER_VIEW_BANK_ACCOUNT`), PRODUCT (incl. `PRODUCT_VIEW_COST`, `PRODUCT_VIEW_STOCK`), INVENTORY, PURCHASE, QUOTATION, INVOICE (VIEW / CREATE / CANCEL / DISCOUNT_OVERRIDE), PAYMENT, EXPENSE, PROJECT, LABOUR, COUPON, REPORT, SETTINGS, DATA_RESET and `DEVELOPER_INSPECT` (held by no default role).

How a permission is enforced, end to end

- **Backend, the real gate:** `@PreAuthorize("hasAuthority('INVOICE_CREATE')")` on the controller method; the authorities come from the JWT, which was built from `role_permission` at login.
- **Frontend, for a sensible UI only:** `RequirePermission` hides routes and `PermissionGate` hides buttons the caller cannot use. Removing them would change nothing about what the API allows.
- **Refusals are recorded:** `RestAccessDeniedHandler` writes an `ACCESS_DENIED` row to `security_audit_log`.
- **Platform admins are a different identity** with their own table, tokens, MFA and audit log; a tenant-facing controller never calls into `platformadmin`.

7. Repository map — where everything lives

```
hardware-erp/
├─ CLAUDE.md                      rules for every task; which guide to load
├─ RESUME_POINT.md                where the last session stopped
├─ README.md, MASTER_PROJECT_STATUS.md, AUDIT_GAP_REPORT.md, IMPROVEMENT_OPPORTUNITIES.md
├─ docker-compose.yml            local PostgreSQL 16
├─ docker-compose.selfhosted.yml the whole stack for a shop's own machine
├─ render.yaml                   cloud deployment manifest
├─ scripts/                      backup-db.sh, restore-db.sh, run-local.sh, run-cloud.*, new-secret.ps1
├─ registry/                     static_check.py, check_registry.py, registry.json (need python3)
├─ project-knowledge/            the registries – source of truth (section 10)
├─ .claude/guides/               coding, business, database, security, testing, architecture rules
├─ docs/                         DEPLOYMENT*.md, MANUAL_WHATSAPP.md, SECURITY_AUDIT.md, Postman, this
overview
├─ backend/                      Maven project
│   └─ pom.xml
│       └─ src/main/java/com/hardware/erp/... src/main/resources/{application*.yaml, db/migration,
db/seed}
│           src/test/java/...          *Test (unit, Mockito) and *IT (Testcontainers
PostgreSQL)
└─ frontend/                     Vite project
    ├── package.json, vite.config.ts, tailwind.config.*, tsconfig*.json
    ├── src/                      (section 9)
    └─ tests/                     Playwright suites + run.mjs, run against the built dist/
```

Backend modules (30 packages)

Package under com.hardware.erp	Owns
auth	Login, refresh, logout, MFA, users, roles, permissions, sessions, security audit log.
tenant	Tenant row, settings, branding media, bank accounts, subscription tier, self-registration (CR-028), data reset (CR-067).
product	Category, brand, product, product image, CSV import, price history.
inventory	stock and stock_movement; StockService.applyMovement is the only writer.
supplier	Suppliers, contacts, encrypted bank details, soft delete + restore.
purchase	Purchases, items, payments, documents, bill import with preview.
customer	Customers and credit limits.
quotation	Quotations and conversion to invoice.
salesorder	Sales orders; conversion to challan or invoice.
deliverychallan	Delivery challans; stock out; conversion to invoice.
invoice	Invoices, items, payments, PDF, email/WhatsApp send.
creditnote	Credit notes; stock back in.
coupon	Coupons and product restrictions.
project	Projects, work types, materials, expenses, payments, profitability.

Package under com.hardware.erp	Owns
labour	Workers, attendance, wages.
expense	Expense categories, business expenses, receipts.
report	Period reports and GSTR-1, rendered via export/.
dashboard, analytics	Aggregations over invoice, stock and payments for the home screen and analytics page.
notification	Email (SMTP / SendGrid), SMS (Twilio), WhatsApp (per-tenant Cloud API) with a delivery log.
export	PDF and XLSX rendering; owns no entity.
ai	Read-only tools over the modules above for the assistant.
supportticket	Support desk.
legal	user_consent entity only (no controller yet).
developer	Diagnostics endpoints, double-gated.
platformadmin, billing	Cross-tenant console and Razorpay subscriptions — deliberately outside tenant isolation, with its own identity and audit log.

Frontend modules (25 folders under src/modules)

Module	Screens and responsibilities
auth	Login, MFA, forgot/reset password, first-run password change, sessions.
onboarding	Permission-filtered product tour and per-page first-visit tips (CR-075).
dashboard	KPI cards with sparklines, sales trend, sales by category, quick actions, recent actions (CR-082/084).
tenant / settings	Shop profile, logo, signature, bank accounts, UPI QR, WhatsApp connection, subscription, data reset.
product	Categories, brands, products, images, CSV import (preview → confirm), price history.
inventory	Stock levels, movements, adjustments, low-stock view.
supplier	Supplier wizard, contacts, bank details behind SUPPLIER_VIEW_BANK_ACCOUNT.
purchase	Purchase entry, bill import, purchase payments, documents.
customer	Customer CRUD with credit limit and map address picker.
quotation	Quotation create / convert to invoice.
invoice	Invoice wizard, detail, cancel, PDF, WhatsApp send.
payment	Standalone receipts view (CR-027).
coupon	Coupon management.
project	Projects, materials, expenses, payments, profitability, rooftop calculator.

Module	Screens and responsibilities
labour	Workers, attendance, wages, project labour cost.
expense	Expense categories and the overhead ledger with receipts.
report	Day Book, Receivables Ageing, Stock Valuation, Purchase Register, GST Summary, GSTR-1 — PDF/XLSX (CR-086/087).
activity	Activity-log viewer (business changes).
notification	Contact-admin form; message log.
support	Support tickets to the platform team.
ai	Assistant chat over the tenant's own data (Anthropic-only, key optional).
tools	Utilities such as the rooftop calculator.
developer	Diagnostics, double-gated: environment and DEVELOPER_INSPECT permission (CR-045).
platform-admin	The cross-tenant console: tenants, incidents, feature flags, system health, billing.

8. Backend — foundation files and the anatomy of one module

These files are shared by every module. A module never ships its own JWT handling, error envelope, money formatting or tenant lookup — it imports these.

File (under backend/src/main/...)	Why it exists
<code>HardwareErpApplication.java</code>	Spring Boot entry point. One JAR, one process.
<code>application*.yaml</code>	Profiles: local, dev, test, prod, plus cloud / selfhosted layered on prod (CR-059). Environments are profiles, never branches.
<code>db/migration/V1..V56__*.sql</code>	Flyway schema history. An applied migration is never edited; a change is a new version (hard rule 1).
<code>db/seed/V900..V904__*.sql</code>	Dev/test seed data only: owner user, sample supplier, products, categories, document sequences.
<code>common/entity/BaseEntity.java</code>	id, created_at/by, updated_at/by, version for every entity.
<code>common/dto/ApiResponse, PageResponse, ErrorResponse</code>	The one response envelope { success, message, data, timestamp } every endpoint returns.
<code>common/exception/GlobalExceptionHandler.java</code>	Maps BusinessException / ResourceNotFound / Duplicate / RateLimit to the right HTTP status and envelope.
<code>common/web/RequestCorrelationFilter.java</code>	Puts an X-Request-ID on every request and log line.
<code>common/activity/ActivityLogWriter, ActivityLogService</code>	Writes activity_log rows (created / updated / cancelled, with before/after JSON).
<code>common/sequence/DocumentSequence.java</code>	Entity behind the race-free document number allocator (CR-041).
<code>common/idempotency/IdempotencyService</code>	Replays a stored response for a repeated Idempotency-Key (CR-051).
<code>common/security/FieldEncryptor.java</code>	AES field encryption for supplier bank details, WhatsApp and Razorpay secrets.
<code>common/util/IndianCurrencyFormat, LineDiscount, Gstin, PhoneNumberNormalizer</code>	Money in paise formatted with Indian digit grouping; discount maths rounded once; GSTIN checksum; phone normalisation.
<code>common/validation/ValidGstin + GstinValidator</code>	Bean-validation annotation with the Modulo-36 GSTIN checksum, on every GSTIN request field.
<code>security/SecurityConfig.java</code>	Filter chain, stateless sessions, public paths, method security on.
<code>security/JwtAuthenticationFilter, JwtService, JwtProperties</code>	Reads the Bearer token, validates it, populates SecurityContext with user id, tenant id and permissions.
<code>security/RefreshTokenCookieService.java</code>	Sets / clears the HttpOnly refresh cookie; Secure flag depends on deployment mode.
<code>security/SecurityUtils.java</code>	<code>currentUserId()</code> , <code>currentTenantId()</code> , <code>requireCurrentTenantId()</code> — the only source of tenant scope.
<code>security/ratelimit/RateLimitFilter, RateLimitService</code>	Bucket4j + Caffeine in-process limits on login, refresh, forgot-password, tenant registration.
<code>security/totp/TotpService.java</code>	TOTP MFA enrolment and verification.
<code>security/captcha/</code>	Captcha check on public endpoints.

File (under backend/src/main/...)	Why it exists
config/DeploymentMode, DeploymentModeGuard	CLOUD vs SELF_HOSTED; refuses to start a self-hosted process pointed at the managed database (CR-059).
config/JwtSecretGuard.java	Refuses to start with a weak or default JWT secret.
config/BootstrapOwnerInitializer.java	Creates the first owner account from environment on an empty database.
config/OpenApiConfig.java	springdoc — Swagger UI at /api/swagger-ui.html.
config/AsyncConfig, JpaAuditingConfig	Thread pool for @Async notifications; fills created_by / updated_by.

Anatomy of one module: invoice

Every business module has the same seven folders, in the same order the code is written (CLAUDE.md, "specification change" steps 3–4). Reading `invoice` teaches you all of them.

invoice/...	Role
entity/Invoice.java, InvoiceItem.java, Payment.java, InvoiceStatus.java, PaymentMethod.java	JPA entities, one per table; status is a Java enum stored as VARCHAR(20).
repository/InvoiceRepository.java ...	Spring Data. Every finder takes tenantId — findByIdAndTenantId, findAllByTenantId(pageable).
dto/InvoiceRequest.java, InvoiceResponse.java, InvoiceSummaryResponse.java ...	Records with Bean Validation. Field names mirror the columns in camelCase; money fields end in Paise.
mapper/InvoiceMapper.java	Entity ↔ DTO, no business logic.
service/InvoiceService.java + service/impl/InvoiceServiceImpl.java	The rules: credit-limit check, coupon, stock movement, document number, activity log, async notification — inside one @Transactional.
pdf/InvoicePdfService.java	Renders the invoice PDF with the tenant's logo, signature, bank account and UPI QR.
controller/InvoiceController.java, PaymentController.java	Thin. @PreAuthorize(PermissionCode.X) on every method; returns ApiResponse.

What happens inside `InvoiceServiceImpl.create()`, in one transaction: resolve the customer and check the credit limit → resolve each product and price the line (`LineDiscount`, rounded to whole paise once) → apply a coupon if given → allocate the invoice number from `document_sequence` under a row lock → save → `StockService.applyMovement(-qty)` per line → record coupon usage → write `activity_log` → after commit, `@Async` notification. If any step throws, none of it happened.

Tests

*Test classes are unit tests with Mockito; *IT classes start a real PostgreSQL 16 in Testcontainers and exercise the HTTP layer with MockMvc, including tenant isolation (`docs/MULTI_TENANT_SECURITY_TEST.md`). 114 test classes today, 36 of them integration. Last full green run: 599 unit + 251 integration, `mvn -o clean verify`.

9. Frontend — foundation files and the anatomy of one module

File (under frontend/src/...)	Why it exists
main.tsx, App.tsx, index.css	Bootstraps React, providers (theme, query, auth), Tailwind layers and the CSS design tokens.
routes/index.tsx	Every route. Auth pages under AuthLayout; the app under AppLayout; /platform-admin/* separately.
routes/ProtectedRoute.tsx, RequirePermission.tsx	Redirect to /login without a session; hide a route the caller has no permission for.
services/apiClient.ts	The single Axios instance: Bearer header, refresh-on-401 once, 90 s timeout for Render cold starts, error normalisation.
services/tokenStorage.ts	Access token in memory only (hard rule 9).
services/apiBaseUrl.ts, deploymentConfig.ts	Which backend to talk to; CLOUD / SELF_HOSTED flags from GET /deployment-config.
services/platformAdminApiClient.ts, platformAdminTokenStorage.ts	Separate client and token for the cross-tenant console.
layouts/AuthLayout.tsx, AuthHeroBackdrop.tsx	The approved sign-in design (CR-081): 50/50 hero, AuthCard shell on all six auth screens.
layouts/AppLayout.tsx, Sidebar.tsx, AppChromeProvider.tsx	The app shell (CR-082): folding nav groups, shop card, header with ⌕ search, ? tour, theme toggle.
layouts/MobileTabBar.tsx, MobileMoreMenu.tsx	Phone shell — native-feeling tab bar (CR-061/062).
theme/ThemeProvider, ColorThemeProvider, colorThemes.ts	Eleven colour themes × light/dark, default emerald. Every colour is a token (hard rule 11).
theme/themeScope.ts, useThemeScope.ts	Per-user client-side scope for theme, tour-seen and first-visit tips.
shared/components/	PageHeader, StatusBadge, Pagination, ConfirmDialog, ImageUpload, AddressMapPicker, GlobalSearch, WhatsAppButton, table/, ui/ (shadcn).
shared/hooks/	useAsyncData, useAsyncList (pagination + filters), useDebounceValue, useAuthenticatedImage, useEnterAdvances.
shared/lib/gstin.ts, geocoding.ts, utils.ts	GSTIN checksum mirrored from the backend; OpenStreetMap geocoding for the address picker.
shared/types/api.ts	ApiResponse, ApiError, PageResponse — the TypeScript twin of common/dto.

Anatomy of one module: modules/invoice

modules/invoice/...	Role
types/index.ts	Mirrors the backend DTOs field-for-field (InvoiceRequest, InvoiceResponse, InvoiceStatus).
validation/schemas.ts	Zod schemas mirroring the Bean Validation rules, so the form rejects what the API would reject.

modules/invoice/...	Role
services/invoiceService.ts	Typed calls through apiClient: list, get, create, cancel, record payment, send WhatsApp.
constants/index.ts	Status labels, colours-by-token, table column definitions.
components/InvoiceStatusBadge, ProductPicker, CreditLimitWarning	Module-local UI pieces.
forms/InvoiceWizard.tsx	The four-step create flow: customer → lines → discount/coupon → review.
pages/InvoiceListPage, InvoiceCreatePage, InvoiceDetailPage	Routed screens; each is wrapped by RequirePermission in routes/index.tsx.

The contract discipline: `types/index.ts` mirrors the Java DTO exactly (a `Long totalPaise` is a `number totalPaise`, never `total`), and `validation/schemas.ts` mirrors the Bean Validation annotations. When a screen shows the wrong thing but Postman shows the right JSON, the bug is in the frontend only — and vice versa. CLAUDE.md requires every bug fix to begin by naming that scope.

Design system rules

- **Never hardcode a colour** (hard rule 11): eleven themes × light/dark from `theme/colorThemes.ts`; components use tokens such as `bg-primary`, `text-muted-foreground`, `--sidebar`.
- **Never draw data that does not exist** (hard rule 12): dashboard sparklines and deltas are measured (CR-084), and an empty state says so.
- **A visible redesign is approved as a render first, then coded exactly** (hard rule 13) — the sign-in page (CR-081) and dashboard (CR-082) were built that way.
- Inline "+ Add new X" (category, brand, expense category) uses one shared pattern; BUG-FE-007 documents the Radix Select quirk it must handle.
- Import flows are always *preview* → *confirm* → *import*, never upload-and-insert (product CSV, supplier bill).

10. Registries and documentation — the source of truth

Decisions live in files, not in anyone's memory. A new contributor (human or AI) reads `CLAUDE.md`, loads the guide for the task, then the registries the task touches. Nobody should have to re-explain a recorded decision.

File	Contains
<code>CLAUDE.md</code>	The rules that apply to every task and which guide to load. Read first, every session.
<code>.claude/guides/*.md</code>	coding-rules, business-rules, database-rules, security-rules, testing, architecture — loaded per task.
<code>project-knowledge/PROJECT_REGISTRY.md</code>	Stack, naming law, module status, deployment modes.
<code>project-knowledge/DATABASE_REGISTRY.md</code>	Every table, column and constraint, with the CR that added it.
<code>project-knowledge/API_REGISTRY.md</code>	Every endpoint and the permission it requires.

File	Contains
project-knowledge/SECURITY_REGISTRY.md	Auth design, token rules, rate limits.
project-knowledge/CHANGE_REQUEST_REGISTRY.md	CR-001 onward: every approved design change and why.
project-knowledge/BUG_REGISTRY.md	Every bug, its layer, root cause and regression test.
project-knowledge/PROJECT_SKILLS.md	Lessons learned — mistakes not to repeat.
project-knowledge/FEATURE_REGISTRY.md, MODULE_DEPENDENCY_MAP.md	Feature status; the real import graph between modules.
RESUME_POINT.md	Exactly where the last session stopped.
registry/static_check.py, check_registry.py	Structure and naming drift checks (need python3).
docs/DEPLOYMENT*.md, MANUAL_WHATSAPP.md, SECURITY_AUDIT.md	Operations: Render + Vercel + Supabase, self-hosted Docker, WhatsApp setup, audit findings.
docker-compose.yml, docker-compose.selfhosted.yml, render.yaml	Local PostgreSQL; the self-hosted stack; the cloud deployment manifest.
scripts/	backup-db.sh, restore-db.sh, run-local.sh, run-cloud.sh/.ps1, new-secret.ps1.
frontend/tests/run.mjs + suites	Playwright suites run against the built dist/.

The shape of every change

1. Name the layer at fault: `SCOPE: FRONTEND ONLY / BACKEND ONLY / BOTH`.
2. Claim a CR or BUG number in the registry *before* writing code (other sessions allocate numbers too — grep first).
3. For a specification change: registry → new Flyway migration → entity → repository → DTO → mapper → service → controller → tests → `types/index.ts` → validation schema → service/form/page → Postman → registries.
4. Fix same-root-cause defects found on the way in the same commit; propose anything bigger as its own CR.
5. Verify in isolation, quote the numbers, update `RESUME_POINT.md`, commit with Conventional Commits (feat: fix: docs: ...), one concern per commit, body says *why*.

11. Conventions that never bend

Naming law — one concept, one name, database → entity → DTO → JSON → TypeScript

Only transformation	snake_case → camelCase. credit_limit_paise is creditLimitPaise everywhere.
Tables	singular snake_case; line tables <parent>_item; app_user because user is reserved.
Primary key	column <table>_id; Java field always id.

Foreign key	column <code><target>_id</code> ; the entity holds the object, the DTO holds <code><target>Id</code> .
Money	<code>BIGINT</code> paise, never float or double. <code>BigDecimal</code> only inside <code>LineDiscount</code> / <code>IndianCurrencyFormat</code> , rounded back to paise once.
Unit rates / quantities	<code>DECIMAL(18,6)</code> / <code>DECIMAL(18,4)</code> — a ₹875 box of 1000 screws is ₹0.875 each.
Status	<code>VARCHAR(20)</code> + CHECK constraint; never <code>TINYINT</code> , never a native ENUM type.
Timestamps / dates / booleans	<code>TIMESTAMP(3)</code> suffix <code>_at</code> ; <code>DATE</code> suffix <code>_date</code> ; booleans without <code>is_</code> .
API paths	<code>/api/v1/<plural-kebab-noun></code> ; verbs only under <code>/auth</code> .
Banned aliases	<code>name</code> , <code>phoneNo</code> , <code>phoneNumber</code> , <code>emailId</code> , <code>gstNumber</code> , <code>UserHelper</code> , <code>UserUtil</code> ...

Hard rules

1. Never edit an applied Flyway migration; add a new version.
2. Hibernate is `ddl-auto: validate`. Never `update`.
3. PostgreSQL only. No MySQL syntax.
4. Seed data in `db/seed/`, dev and test profiles only.
5. No self-registration of users; the owner creates accounts. (New-tenant sign-up is a separate, rate-limited endpoint — CR-028.)
6. Authorization is permission-based, never `hasRole('OWNER')`.
7. Users and suppliers are soft-deleted; financial records reference them forever.
8. Security events → `security_audit_log`; business changes → `activity_log`.
9. Access token in memory only. Never `localStorage`.
10. Never claim a build passes without running it.
11. Never hardcode a colour.
12. Never draw data that does not exist.
13. A visible redesign is approved as a render first, then coded exactly.

Git

`main` (tagged releases) ← `develop` ← `feature/*`, `bugfix/*`; `hotfix/*` from `main` back into both. Merge with `-no-ff`. Runtime environments are Spring profiles, never branches. Several sessions work the same checkout at once, so files are staged by explicit path and CR numbers are claimed in the registry first.

12. How to run and verify

```

docker compose up -d                                # PostgreSQL 16 on localhost
cd backend && mvn spring-boot:run -Dspring-boot.run.profiles=local
cd frontend && npm install && npm run dev              # http://localhost:5173 → /api on :8080
Swagger: http://localhost:8080/api/swagger-ui.html

# verification (quote the numbers, never assume)
cd backend && mvn -o clean verify                      # unit + Testcontainers ITs (needs
Docker)
cd frontend && node ./node_modules/typescript/bin/tsc -b --force
cd frontend && node ./node_modules/vite/bin/vite.js build
cd frontend && node tests/run.mjs                      # Playwright, against dist/
python3 registry/static_check.py                      # structure + naming drift

# this document

```

```
node docs/project-overview/build.mjs  
migrations
```

```
# regenerates the PDF from the
```

On a Windows machine with Git Bash, npm script shims can fail (`"node"` is not recognized) — call the tools directly as shown. `docker` from Git Bash needs an isolated `DOCKER_CONFIG` holding an empty `config.json`.

Appendix A — tables not drawn in the ER diagram

These 28 tables exist in the same schema and follow the same rules; they were left out of the drawing only because they hang off `tenant`, `app_user` or each other and would add lines without adding understanding. The platform-console tables are the one deliberate exception to tenant isolation.

Branding & documents (tenant-owned media, stored as bytea)

Table	Since	Cols	Tenant	Foreign keys	What it holds
<code>tenant_logo</code>	V11	5	yes	<code>tenant_id</code> → <code>tenant</code>	Shop logo printed on invoices.
<code>tenant_signature</code>	V11	5	yes	<code>tenant_id</code> → <code>tenant</code>	Authorised signature image for invoices.
<code>tenant_upi_qr</code>	V13	5	yes	<code>tenant_id</code> → <code>tenant</code>	Shop-level UPI QR code.
<code>tenant_bank_account</code>	V22	14	yes	<code>tenant_id</code> → <code>tenant</code>	Bank accounts shown on invoices for payment (CR-036).
<code>tenant_bank_account_qr</code>	V22	10	—	<code>bank_account_id</code> → <code>tenant_bank_account</code>	UPI QR per bank account.
<code>user_avatar</code>	V11	5	—	<code>user_id</code> → <code>app_user</code>	Profile picture.

Notifications & messaging

Table	Since	Cols	Tenant	Foreign keys	What it holds
<code>notification_log</code>	V14	12	yes	<code>tenant_id</code> → <code>tenant</code>	Every email / SMS / WhatsApp attempt with its status (CR-027, CR-074).
<code>tenant_whatsapp_connection</code>	V45	15	yes	<code>tenant_id</code> → <code>tenant</code>	The tenant's own WhatsApp Business API credentials, encrypted (CR-056).

Security, audit & compliance

Table	Since	Cols	Tenant	Foreign keys	What it holds
<code>security_audit_log</code>	V1	12	—	—	Login, logout, lockout, MFA and ACCESS_DENIED events (hard rule 8).
<code>activity_log</code>	V3	16	yes	—	Business change history with before/after values (CR-015); <code>tenant_id</code> added in V55.
<code>password_reset_token</code>	V1	7	—	<code>user_id</code> → <code>app_user</code>	Single-use forgot-password tokens.
<code>user_backup_code</code>	V52	5	—	<code>user_id</code> → <code>app_user</code>	Hashed MFA recovery codes.
<code>user_consent</code>	V28	11	yes	<code>tenant_id</code> → <code>tenant</code> <code>user_id</code> → <code>app_user</code>	Which terms/privacy version a user accepted.

Support desk

Table	Since	Cols	Tenant	Foreign keys	What it holds
support_ticket	V47	12	yes	tenant_id → tenant user_id → app_user	A tenant's support request to the platform team.
support_ticket_message	V47	8	—	support_ticket_id → support_ticket	Messages in a ticket thread.

Cross-tenant Platform Admin console (outside tenant isolation, own auth)

Table	Since	Cols	Tenant	Foreign keys	What it holds
platform_admin	V39	18	—	created_by → platform_admin updated_by → platform_admin	Operator accounts for the cross-tenant console (CR-054). Own auth, own refresh tokens, own MFA.
platform_admin_backup_code	V39	5	—	platform_admin_id → platform_admin	MFA recovery codes for platform admins.
platform_admin_refresh_token	V39	11	—	platform_admin_id → platform_admin	Refresh tokens for platform admins.
platform_audit_log	V39	10	—	—	What platform admins did, across tenants.
platform_incident	V46	13	—	—	Operational incidents shown on the platform status page.
feature_flag	V48	9	—	—	Platform-wide feature switches.
job_execution_log	V46	7	—	—	Scheduled-job runs (backups, snapshots).
platform_subscription_order	V49	11	yes	tenant_id → tenant	A tenant's subscription order (CR-057).
platform_subscription_payment	V49	12	—	subscription_order_id → platform_subscription_order	Razorpay payment against an order.
platform_tenant_export	V50	9	yes	tenant_id → tenant	A requested tenant data export / backup (CR-057).
platform_razorpay_config	V51	9	—	—	Razorpay keys, encrypted.
subscription_coupon	V20	16	yes	tenant_id → tenant	Trial coupons that grant a plan free for a period (CR-032).

Cross-cutting infrastructure

Table	Since	Cols	Tenant	Foreign keys	What it holds
idempotency_record	V34	8	yes	tenant_id → tenant	Stored response for an Idempotency-Key so a retried POST cannot create a second document (CR-051).